Módulo: Entornos de Desarrollo.
Unidad de Trabajo Nº4: Optimización y documentación
Actividad Integradora UTNº4: Optimización y documentación.

1. **Objetivo general.**
   Que el alumnado sea capaz de refactorizar, documentar y versionar código Java de forma integrada, combinando las técnicas de optimización de código, la generación de documentación con JavaDoc y el uso del control de versiones con Git y GitHub.
2. **Metodología**.
   Cada alumno refactorizará y documentará utilizando diferentes IDEs.
3. **Descripción**.

La actividad se divide en tres partes que deben realizarse en orden:

• Primera Parte – Refactorización y Documentación de código existente.
• Segunda Parte – Desarrollo de la clase Cubo con JavaDoc.
• Tercera Parte – Control de versiones con Git y GitHub (local, remoto y colaborativo).

Primera parte – Refactorización y Documentación

Código Original

El Introduce el siguiente código de Java,

```
public class Main {
   public static void main(final String[] args) {
      int variableA = 0; int variableB = 15; int variableC = 20; int valor = 0;
         do { variableC = 10 + variableB; valor = valor + variableC;  valor++; if (valor < 10)
{ variableA = 15;  valor = valor – 1; } else { variableA = 10; valor = valor – 1; }
         int aux; aux = valor; variableA = aux; } while (valor < 100); }
}
```

Tareas de Refactorización

Utilizando las herramientas de refactorización de tu IDE (Eclipse, IntelliJ IDEA o Visual Studio Code), realiza los siguientes cambios::

1. Formatea el código para que cada línea contenga una única instrucción y ajusta correctamente el sangrado (tabulación).
2. Renombra las variables para que sus nombres describan con claridad la función que realizan en el programa.
3. Genera los comentarios JavaDoc de la clase y de cada uno de los métodos.

Entrega: entrega el código reformateado y refactorizado y funcional.

Segunda parte – Clase Cubo con JavaDoc

Descripción

Desarrolla una aplicación Java modular basada en la clase Cubo. La aplicación deberá solicitar al usuario el parámetro necesario para crear el cubo (el lado), calcular su área de la base y su volumen, y presentar los resultados mediante ventanas de diálogo (JOptionPane)..

Requisitos de la Clase Cubo

1. Constructor que reciba como parámetro el lado del cubo.
2. Getters y setters para el acceso al atributo lado.
3. float calcularAreaBase() – calcula y devuelve el área de la base del cubo.
4. float calcularVolumen() – calcula y devuelve el volumen del cubo.
5. Método para solicitar el valor del lado al usuario mediante JOptionPane.
6. Método para imprimir el área del cubo mediante JOptionPane.
7. Método para imprimir el volumen del cubo mediante JOptionPane.
8. En el método main, crea tres cubos y determina cuál tiene el mayor volumen.
9. Prueba el funcionamiento completo de la aplicación.
10. Genera la documentación JavaDoc completa de la clase y todos sus métodos.

Entrega: entrega el código que compila funciona y sin errores.

Tercera parte – Control de Versiones con Git y GitHub

Git Local: Repositorio y Primeros Commits

Configuración inicial de Git

Antes de empezar, configura tu identidad en Git. Abre Git Bash y ejecuta:

```
git config --global user.name "Tu Nombre y Apellido"
git config --global user.email "tu.email@tudominio.com"
git config --global --list
```

Creación del repositorio local

Crea una carpeta para tu proyecto y transfórmala en un repositorio de Git:

```
mkdir AI_ut4_TuNombre
cd AI_ut4_TuNombre
git init
```

Primer ciclo: add → commit

Añade los archivos Java de la Primera y Segunda parte al repositorio y realiza el primer commit:

```
git add Main.java
git status
git commit -m "Añadir clase Main refactorizada con JavaDoc"
```

Realiza al menos un segundo commit con alguna mejora o adición de código:

```
git add Cubo.java
git commit -m "Añadir clase Cubo con metodos de calculo y JavaDoc"
```

```
git log
```

Repositorio Remoto en GitHub

Crear el repositorio en GitHub

1. Inicia sesión en tu cuenta de GitHub (https://github.com).
2. Haz clic en el botón + > New repository.
3. Nombre del repositorio: AI_ut4_TuNombre.
4. Visibilidad: Public.
5. IMPORTANTE: deja desmarcadas las casillas Add a README file y Add .gitignore para evitar conflictos con los archivos locales.
6. Haz clic en Create repository.

Vincular el repositorio local con el remoto

Copia la URL HTTPS del repositorio que te muestra GitHub y ejecuta (sustituye tu-usuario por tu nombre de usuario):

```
git remote add origin https://github.com/tu-usuario/AI_ut4_TuNombre.git
git remote -v
```

Primer push a GitHub

Renombra la rama principal a main (estándar actual) y sube tus commits:

```
git branch -M main
git push -u origin main
```

Si es la primera vez que usas Git con GitHub en este equipo, se abrirá una ventana para autenticarte. Sigue los pasos en el navegador para autorizar la conexión.

Verifica en la web de GitHub que tu archivo aparece en el repositorio junto al historial de commits.

Realiza al menos un segundo push con cambios adicionales:

```
git add .
git commit -m "Añadir clase GestorCubos y pruebas"
git push origin main
```

Git Colaborativo: GitHub Flow

En esta parte trabajarás en equipo (2 personas) siguiendo el flujo de trabajo GitHub Flow, el más utilizado en el sector profesional.

Vamos a modificar el Proyecto de Cubos de la siguiente forma:

- Añadiendo la clase **GestorCubos** que sea responsable de manejar todas las operaciones relacionadas con los objetos de tipo **Cubo**.
  - Un método para inicializar y poblar una lista de cubos.
  - Un método que ordene la lista de cubos por su tamaño (de menor a mayor basándose en su lado/volumen).
  - Un método booleano que reciba dos cubos y determine si el primero cabe exactamente dentro del segundo.

- Modificando la clase **Main** no debe contener lógica de negocio. Su única responsabilidad será.
  - Instanciar la clase **GestorCubos**.
  - Pasar los datos iniciales (crear un par de cubos de prueba).
  - Llamar a los métodos del gestor para demostrar que la ordenación y la comprobación de "encaje" funcionan correctamente.

Antes de desarrollar recuerda añadir a un/a compañero/a como colaborador/a y luego realiza los siguiente:

Primera Feature (El Gestor)

- A partir de la rama **main** actualizada, crea una rama llamada **feature/gestor-cubos**

  git checkout main
  git pull origin main
  git checkout -b feature/gestor-cubos

- Desarrolla únicamente la clase **GestorCubos** en esta rama.
- Haz commit, súbela al repositorio remoto y abre un **Pull Request**.

  git add GestorCubos.java

git commit -m "Añade clase GestorCubos con lógica de ordenación y encaje"
git push -u origin feature/gestor-cubos

- Asigna el PR a tu compañero/a para que lo valide. Una vez aprobado, intégrelo (merge) en **main.**

Segunda Feature (El Refactor):

- Vuelve a tu entorno local, actualiza tu rama main (para obtener el Gestor recién fusionado).
- Crea una nueva rama llamada **feature/refactor-main**.

git checkout main
git pull origin main # Esto descarga el GestorCubos que Elena acaba de fusionar
git checkout -b feature/refactor-main

- Modifica la clase **Main** para usar el nuevo Gestor.
- Haz *commit*, súbela y abre un segundo **Pull Request**.

git add Main.java
git commit -m "Refactoriza Main para delegar la ejecución en GestorCubos"
git push -u origin feature/refactor-main

- Espera la revisión y aprobación de tu compañero/a antes de fusionar esta rama también a main.
- Para terminar y tener tu entorno local perfectamente sincronizado con el trabajo finalizado.

git checkout main
git pull origin main

Entrega

- Código funcional sin errores. Con la clase Main que sólo actúa como punto de entrada.
- El historial de Git muestra claramente dos ramas distintas que fueron creadas, trabajadas y fusionadas de vuelta a `main` de manera independiente.